Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Разработка классов TBitField и TSet»

Выполнил:
студент группы 3825Б1ФИсп1
Чернов А.А.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 7
4. Результаты экспериментов и тестирования 10
Заключение 13
Литература 14
Приложение 15

Введение

Битовые поля и множества — классические структуры данных, широко применяемые в задачах на программирование. Битовое поле позволяет компактно хранить набор логических значений, используя один бит на каждый элемент, что даёт существенную экономию памяти по сравнению с массивом **bool**. Множества, в свою очередь, используются для представления неупорядоченных коллекций уникальных элементов и поддерживают теоретико-множественные операции: объединение, пересечение, разность, проверку принадлежности.

1. Постановка задачи

Цель работы — реализовать на C++ два класса: TBitField (битовое поле) и TSet (множество на основе характеристического вектора).

Требуется реализовать для класса TBitField:

- Конструктор TBitField(int len) — создаёт поле на len бит, выделяет память, обнуляет её.
- Конструктор копирования и деструктор — с корректной работой с динамической памятью (глубокая копия, освобождение).
- Оператор присваивания =.
- Доступ к битам: GetLength(), SetBit(n), ClrBit(n), GetBit(n).
- Операторы сравнения ==, !=.
- Побитовые операции |, &, ~.
- Операторы ввода-вывода >>, << (как строка из 0 и 1).

Требуется реализовать для класса TSet:

- Конструкторы: TSet(int mp), копирования, из TBitField.
- Преобразование типа operator TBitField().
- Работа с элементами: InsElem(x), DelElem(x), IsMember(x), GetMaxPower().
- Операторы: ==, !=, =, + int, - int, + TSet, * TSet, ~.
- Операторы ввода-вывода.

Дополнительные требования:

- Все методы располагаются в .h (header) файлах.
- Написать модульные тесты (Google Test) на основные сценарии.

2. Руководство пользователя

Запуск программы. Программа собирается в Visual Studio 2026 как консольное приложение (Lab1.exe) и запускается стандартным способом (F5 или двойным кликом по .exe).

Назначение. Программа демонстрирует работу классов TBitField (битовое поле) и TSet (множество на основе характеристического вектора).

Сценарии, показанные в программе:

1. **Создание множества и добавление элементов.** Создаётся множество s1 с универсумом {0..7}. В него добавляются элементы 2, 5 и 7.
2. **Ввод элементов множества.** Пользователю предлагается ввести номера элементов через пробел. Программа считывает строку и добавляет указанные элементы в новое множество s2.
3. **Обработка ошибки выхода за границы.** Демонстрируется попытка добавить элемент 100 в множество с универсумом {0..7}. Программа перехватывает исключение std::out_of_range и выводит сообщение об ошибке, не завершая аварийно свою работу.
4. **Обработка ошибки несовпадения универсумов.** Демонстрируется попытка сравнить множества с разной мощностью (8 и 16). Программа перехватывает исключение std::invalid_argument и выводит соответствующее сообщение.

Формат вывода. Битовое поле выводится в виде строки из 0 и 1 (старший бит слева). Для множества формат вывода был изменён на более наглядный: оно выводится в виде перечисления своих элементов в фигурных скобках (например, {2, 5, 7}). Ввод элементов множества также осуществляется через их числовые номера, разделённые пробелами.

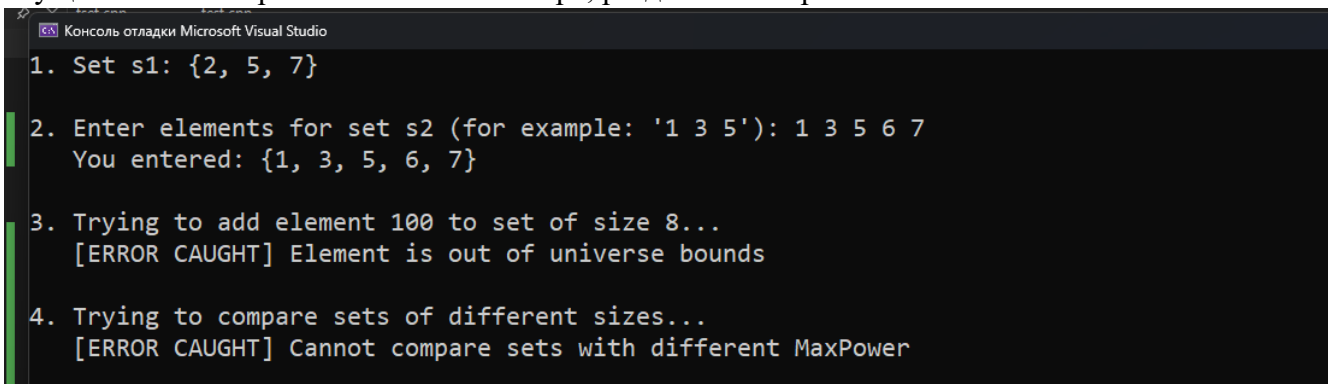


Рис. 1. Результат работы программы Lab1.exe

Примечание. Основная проверка корректности реализована через модульные тесты Google Test. Результаты приведены в разделе 4.

3. Руководство программиста

3.1. Описание структуры программы

Модуль и его назначение:

- tbitfield.h – класс битового поля
- tset.h – класс множества на основе битового поля
- main.cpp – демонстрационная программа
- test.cpp – Google тесты

Модуль tbitfield.h содержит:

- Класс TBitField с приватными полями BitLen, pMem, MemLen.
- Приватные вспомогательные методы GetMemIndex, GetMemMask.
- Публичные конструкторы, деструктор, методы доступа к битам, перегруженные операторы.

Модуль tset.h содержит:

- Класс TSet с приватными полями MaxPower, BitField.
- Публичные конструкторы, методы работы с элементами, теоретико-множественные операторы.
- Все операции делегируются внутреннему объекту BitField — композиция вместо наследования.

Тесты вынесены в отдельный проект lab1_tests, который ссылается на основной проект Lab1. Используется библиотека Google Test. Тесты сгруппированы по TestCaseName: TBitFieldConstructor, TBitFieldBits, TBitFieldCopy, TBitFieldCompare, TBitFieldAssign, TBitFieldOperations, TBitFieldInputOut, TSetConstructor, TSetElements, TSetCopy, TSetCompare, TSetAssign, TSetOperations.

3.2. Описание структур данных

Класс TBitField

Назначение: хранение и обработка последовательности битов фиксированной длины.

Приватные поля - тип:

- BitLen – int – длина поля это количество значащих битов
- pMem – TELEM* - Указатель на массив элементов типа TELEM (где TELEM = unsigned int = 32 бита)
- MemLen – int – Количество элементов в массиве pMem, вычисляется как (BitLen +31) /32

Приватные методы:

- GetMemIndex(n) – Индекс элемента pMem, в котором хранится бит n (формула: n/32)
- GetMemMask(n) – Маска с единицей в позиции бита n внутри элемента (формула: 1 << (n%32))

Класс TSet

Назначение: представление множества целых чисел из универсума {0, 1, ..., MaxPower-1} в виде характеристического вектора.

Приватные поля - тип:

- MaxPower – int – мощность универсума
- BitField – TbitField – характеристический вектор

Композиция: TSet содержит объект TBitField как поле. Все операции над множествами реализуются через вызовы методов и перегруженных операторов TBitField.

3.3. Описание алгоритмов

Работа с памятью

Конструктор TBitField(int len). Вычисляется MemLen = (len + 31) / 32 — округление вверх при целочисленном делении. Выделяется массив new TELEM[MemLen], все элементы обнуляются циклом. Обнуление обязательно: new[] не инициализирует память, там может быть мусор.

Конструктор копирования. Глубокая копия: копируются BitLen и MemLen, выделяется новая память, элементы копируются поэлементно. Простое pMem = bf.pMem дало бы два объекта с общим массивом — при уничтожении был бы двойной delete[].

Деструктор. Освобождает память через delete[] pMem (именно delete[], потому что выделяли new[]).

Оператор присваивания. Три шага:

1. Проверка самоприсваивания &bf == this — иначе после освобождения памяти мы бы читали из неё же.
2. delete[] pMem — освобождение старой памяти, иначе утечка.
3. Копирование полей, выделение новой памяти, поэлементное копирование.

Доступ к битам

GetMemIndex(n). Индекс элемента: n / 32. Например, бит 33 лежит в pMem[1].

GetMemMask(n). Маска: 1 << (n % 32). Позиция внутри элемента — остаток от деления на 32.

SetBit(n). pMem[i] = pMem[i] | mask. Операция | с единицей на нужной позиции устанавливает бит в 1, не трогая соседние (X | 0 = X).

ClrBit(n). pMem[i] = pMem[i] & ~mask. Инвертированная маска имеет 0 на нужной позиции и 1 на остальных. & зануляет нужный бит, не трогая соседние.

GetBit(n). (pMem[i] & mask) != 0. Если бит = 1 — результат ненулевой, приводится к 1. Если 0 — к 0.

Операторы

operator==. Сначала проверяется равенство BitLen — поля разной длины не равны. Затем поэлементное сравнение pMem[i] != bf.pMem[i]. При первом несовпадении — досрочный return 0.

operator!=. Реализован через ==: возвращает инвертированный результат.

operator| и operator&. Результат — новое поле длины min(BitLen, bf.BitLen). Идём по элементам result.MemLen, применяем | или & поэлементно. Взятие min гарантирует, что не выйдем за границы ни одного из массивов.

operator~. Создаётся поле той же длины. Все биты инвертируются через ~pMem[i]. После этого нужно обнулить незначащие биты последнего элемента: они были 0, стали 1. Считаем valid_bit

= BitLen - (MemLen - 1) * 32, строим маску (1 << valid_bits) - 1 и применяем &. Проверка valid_bits < 32 защищает от сдвига на ширину типа.

operator>> и operator<<. Вывод идёт от BitLen-1 до 0 — старший бит слева. Ввод симметричен: первый символ идёт в старший бит. Используется istr.get(c) / ostr << по одному символу. Оба оператора возвращают поток для цепочек.

Класс TSet

Все операции — делегация к TBitField. Своей логики почти нет

TSet(int mp). MaxPower = mp, внутренний BitField создаётся через список инициализации: : BitField(mp). Длина поля = мощности универсума.

Работа с элементами:

- InsElem(x) → BitField.SetBit(x)
- DelElem(x) → BitField.ClrBit(x)
- IsMember(x) → BitField.GetBit(x)

operator+ (int Elem) / operator- (int Elem). Создаётся копия *this, в ней добавляется/удаляется элемент, возвращается копия. Оригинал не меняется.

Для обеспечения безопасности работы с множествами реализован механизм исключений. Если индекс элемента выходит за пределы универсума (меньше 0 или больше/равен MaxPower), методы InsElem, DelElem и IsMember выбрасывают исключение std::out_of_range. Если происходит попытка сравнения или объединения множеств с разной мощностью универсума, выбрасывается исключение std::invalid_argument, так как такие множества являются разными математическими объектами.

Теоретико-множественные операции:

Объединение + (TSet) → BitField | s.BitField

Пересечение * (TSet) → BitField & s.BitField

Дополнение ~ → ~BitField

Результат оборачивается в новый TSet через конструктор преобразования.

operator=. Проверка самоприсваивания, копирование MaxPower, затем BitField = s.BitField — используется готовый operator= класса TBitField.

Ключевые технические решения

Характеристический вектор. Множество хранится как битовое поле — все операции становятся побитовыми и выполняются над 32 элементами за одну инструкцию.

Композиция вместо наследования. TSet содержит TBitField, а не наследуется от него. Это логически верно: множество — не разновидность битового поля, а его использование.

MIN при | и &. Если длины разные — берём минимум. Безопасно, не выходим за границы. Для & теряются только нулевые биты, для | — возможна потеря старших битов длинного поля (в работе предполагается единый универсум).

Чистка хвоста в ~. Единственный метод, где обрабатываются незначащие биты. Если бы их не обнулили, нарушился бы инвариант класса и сломались бы == и последующие ~.

4. Результаты экспериментов и тестирования

Методика тестирования. Для проверки корректности работы классов использована библиотека Google Test. Тесты вынесены в отдельный проект lab1_tests и запускаются через встроенный обозреватель тестов Visual Studio 2026.

Объём тестирования. Написано 33 тестов, сгруппированных в 14 групп (TestCaseName):

Группа	Кол-во	Что проверяется
TBitFieldConstructor	2	Создание пустого поля, округление MemLen вверх
TBitFieldBits	3	SetBit, ClrBit, GetBit, независимость соседних битов
TBitFieldCopy	2	Глубокая копия, независимость копии от оригинала
TBitFieldCompare	2	= = и !=, поля разной длины
TBitFieldAssign	2	Присваивание, самоприсваивание, разная длина
TBitFieldOperations	4	|, &, ~, двойная инверсия
TBitFieldInputOut	3	Ввод, вывод, roundtrip
TSetConstructor	1	Пустое множество
TSetElements	2	InsElem, DelElem, IsMember, сохранение оригинала
TSetCopy	2	Глубокая копия множества
TSetCompare	1	= = и !=
TSetAssign	1	Присваивание и самоприсваивание
TSetOperations	3	Объединение, пересечение, дополнение
TSetExceptions	5	Проверка выброса исключений (выход за границы, разные универсумы)

Ключевые проверяемые сценарии:

Управление памятью. Тесты копирования и присваивания проверяют, что копия независима от оригинала — изменение одного объекта не влияет на другой. Это подтверждает корректность глубокого копирования и отсутствие двойного освобождения памяти.

Самоприсваивание. Отдельный тест a = a проверяет защиту от чтения из освобождённой памяти.

Обработка исключений. Тесты группы TSetExceptions проверяют защиту от некорректных данных. Проверяется, что при попытке добавить/удалить элемент вне диапазона универсума (меньше 0 или больше/равен MaxPower) выбрасывается исключение std::out_of_range. Также проверяется, что операции сравнения и объединения множеств с разной мощностью выбрасывают std::invalid_argument.

Граничные случаи. Тест round_up_if_more_than_32 использует BitLen = 36 — длину, не кратную 32. Тест double_not_returns_original использует BitLen = 100 и проверяет, что ~~a == a — это гарантирует корректную очистку незначащих битов в operator~.

Roundtrip ввода-вывода. Тест in_out_equal проверяет, что если вывести поле в поток, а затем прочитать обратно, получится исходное поле.

Симметрия операций множеств. Тесты union_sets, intersection_sets, complement проверяют, что операции +, *, ~ над TSet дают ожидаемый результат.

Результат. Все 33 теста пройдены успешно. Скриншот результатов приведён на Рис. 2. и Рис. 3.

```
[==========] Running 33 tests from 14 test cases.
[----------] Global test environment set-up.
[----------] 2 tests from TBitFieldConstructor
[ RUN      ] TBitFieldConstructor.empty_field
[       OK ] TBitFieldConstructor.empty_field (0 ms)
[ RUN      ] TBitFieldConstructor.round_up_if_more_than_32
[       OK ] TBitFieldConstructor.round_up_if_more_than_32 (0 ms)
[----------] 2 tests from TBitFieldConstructor (0 ms total)

[----------] 3 tests from TBitFieldBits
[ RUN      ] TBitFieldBits.set_single_bit
[       OK ] TBitFieldBits.set_single_bit (0 ms)
[ RUN      ] TBitFieldBits.set_multiple_bits
[       OK ] TBitFieldBits.set_multiple_bits (0 ms)
[ RUN      ] TBitFieldBits.clear_bit_keeps_others
[       OK ] TBitFieldBits.clear_bit_keeps_others (0 ms)
[----------] 3 tests from TBitFieldBits (0 ms total)

[----------] 2 tests from TBitFieldCopy
[ RUN      ] TBitFieldCopy.copy_really_is_copy
[       OK ] TBitFieldCopy.copy_really_is_copy (0 ms)
[ RUN      ] TBitFieldCopy.changing_copy_doesnt_change_original
[       OK ] TBitFieldCopy.changing_copy_doesnt_change_original (0 ms)
[----------] 2 tests from TBitFieldCopy (0 ms total)

[----------] 2 tests from TBitFieldCompare
[ RUN      ] TBitFieldCompare.compare_equal_and_different
[       OK ] TBitFieldCompare.compare_equal_and_different (0 ms)
[ RUN      ] TBitFieldCompare.different_length_not_equal
[       OK ] TBitFieldCompare.different_length_not_equal (0 ms)
[----------] 2 tests from TBitFieldCompare (1 ms total)

[----------] 2 tests from TBitFieldAssign
[ RUN      ] TBitFieldAssign.assign_and_selfassign
[       OK ] TBitFieldAssign.assign_and_selfassign (0 ms)
[ RUN      ] TBitFieldAssign.assign_different_lengths
[       OK ] TBitFieldAssign.assign_different_lengths (0 ms)
[----------] 2 tests from TBitFieldAssign (1 ms total)

[----------] 4 tests from TBitFieldOperations
[ RUN      ] TBitFieldOperations.bit_or
[       OK ] TBitFieldOperations.bit_or (0 ms)
[ RUN      ] TBitFieldOperations.bit_and
[       OK ] TBitFieldOperations.bit_and (0 ms)
[ RUN      ] TBitFieldOperations.bit_not
[       OK ] TBitFieldOperations.bit_not (0 ms)
[ RUN      ] TBitFieldOperations.double_not_returns_original
[       OK ] TBitFieldOperations.double_not_returns_original (0 ms)
[----------] 4 tests from TBitFieldOperations (1 ms total)
```

Рис.2. Результаты тестов

```
[----------] 3 tests from TBitFieldInputOut
[ RUN      ] TBitFieldInputOut.out_string
[       OK ] TBitFieldInputOut.out_string (0 ms)
[ RUN      ] TBitFieldInputOut.in_string
[       OK ] TBitFieldInputOut.in_string (0 ms)
[ RUN      ] TBitFieldInputOut.in_out_equal
[       OK ] TBitFieldInputOut.in_out_equal (0 ms)
[----------] 3 tests from TBitFieldInputOut (1 ms total)

[----------] 1 test from TSetConstructor
[ RUN      ] TSetConstructor.empty_set
[       OK ] TSetConstructor.empty_set (0 ms)
[----------] 1 test from TSetConstructor (0 ms total)

[----------] 2 tests from TSetElements
[ RUN      ] TSetElements.insert_delete_check
[       OK ] TSetElements.insert_delete_check (0 ms)
[ RUN      ] TSetElements.elem_operators_dont_change_original
[       OK ] TSetElements.elem_operators_dont_change_original (0 ms)
[----------] 2 tests from TSetElements (1 ms total)

[----------] 2 tests from TSetCopy
[ RUN      ] TSetCopy.copies_content
[       OK ] TSetCopy.copies_content (0 ms)
[ RUN      ] TSetCopy.changing_copy_doesnt_change_original
[       OK ] TSetCopy.changing_copy_doesnt_change_original (0 ms)
[----------] 2 tests from TSetCopy (0 ms total)

[----------] 1 test from TSetCompare
[ RUN      ] TSetCompare.equal_and_different
[       OK ] TSetCompare.equal_and_different (0 ms)
[----------] 1 test from TSetCompare (0 ms total)

[----------] 1 test from TSetAssign
[ RUN      ] TSetAssign.assign_and_selfassign
[       OK ] TSetAssign.assign_and_selfassign (0 ms)
[----------] 1 test from TSetAssign (0 ms total)

[----------] 3 tests from TSetOperations
[ RUN      ] TSetOperations.union_sets
[       OK ] TSetOperations.union_sets (0 ms)
[ RUN      ] TSetOperations.intersection_sets
[       OK ] TSetOperations.intersection_sets (0 ms)
[ RUN      ] TSetOperations.complement
[       OK ] TSetOperations.complement (0 ms)
[----------] 3 tests from TSetOperations (1 ms total)

[----------] 5 tests from TSetExceptions
[ RUN      ] TSetExceptions.insert_out_of_bounds
[       OK ] TSetExceptions.insert_out_of_bounds (0 ms)
[ RUN      ] TSetExceptions.delete_out_of_bounds
[       OK ] TSetExceptions.delete_out_of_bounds (0 ms)
[ RUN      ] TSetExceptions.is_member_out_of_bounds
[       OK ] TSetExceptions.is_member_out_of_bounds (0 ms)
[ RUN      ] TSetExceptions.compare_different_sizes
[       OK ] TSetExceptions.compare_different_sizes (0 ms)
[ RUN      ] TSetExceptions.union_different_sizes
[       OK ] TSetExceptions.union_different_sizes (0 ms)
[----------] 5 tests from TSetExceptions (1 ms total)

[----------] Global test environment tear-down
[==========] 33 tests from 14 test cases ran. (14 ms total)
[  PASSED  ] 33 tests.
```

Рис. 3. Результаты тестов (продолжение)

Заключение

В ходе выполнения лабораторной работы были реализованы два класса на языке C++:

Класс TBitField — битовое поле, обеспечивающее компактное хранение последовательности битов. Реализованы:

- корректная работа с динамической памятью (конструктор, деструктор, глубокое копирование, оператор присваивания с защитой от самоприсваивания);
- доступ к отдельным битам без изменения соседних;
- перегруженные операторы ==, !=, |, &, ~;
- операторы ввода-вывода в виде строки из 0 и 1.

Класс TSet — множество на основе характеристического вектора. Реализованы:

- конструкторы (по мощности, копирования, из TBitField);
- операции над элементами (InsElem, DelElem, IsMember);
- теоретико-множественные операции (+, -, *, ~);
- перегруженные операторы сравнения, присваивания, ввода-вывода.
- механизм исключений для защиты от выхода за границы универсума (std::out_of_range) и операций с разными мощностями (std::invalid_argument);
- ввод и вывод элементов в виде их номеров в фигурных скобках (вместо битовых строк).

Ключевые результаты:

Показано, что характеристический вектор позволяет свести теоретико-множественные операции к побитовым — все операции над множествами выполняются через уже готовые операторы TBitField, что упрощает реализацию и ускоряет работу.

Литература

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Документация Google Test: https://google.github.io/googletest/

Приложение

Код main.cpp, показывающий минимальную работу реализованных классов. Полная проверка реализации и всех методов выполнена с помощью Google Tests

main.cpp

```
#include <iostream>
#include "../../../includes/Lab1/tbitfield.h"
#include "../../../includes/Lab1/tset.h"
#include <stdexcept> //исключения

int main() {
    TSet s1(8);
    s1.InsElem(2);
    s1.InsElem(5);
    s1.InsElem(7);

    std::cout << "1. Set s1: " << s1 << std::endl;
    //ожидаем: {2, 5, 7} (а не 00100101 как раньше)

    std::cout << "\n2. Enter elements for set s2 (for example: '1 3 5'): ";
    TSet s2(45);
    std::cin >> s2;
    std::cout << "   You entered: " << s2 << std::endl;

    std::cout << "\n3. Trying to add element 100 to set of size 8..." << std::endl;
    try {
        s1.InsElem(100); //это вызовет ошибку
    }
    catch (const std::out_of_range& e) {
        std::cout << "   [ERROR CAUGHT] " << e.what() << std::endl;
    }

    std::cout << "\n4. Trying to compare sets of different sizes..." << std::endl;
    TSet s3(16);
    try {
        if (s1 == s3) { //это вызовет ошибку (универс из 8 пытаемся сравнить с универсом из 16)
            std::cout << "Equal" << std::endl;
        }
    }
    catch (const std::invalid_argument& e) {
        std::cout << "   [ERROR CAUGHT] " << e.what() << std::endl;
    }

    return 0;
}
```